

Replicating Simple Graph Convolutions

Neha Arora (na458), Sol Jung (sj777), Shashank Kalyanaraman (ssk252), Kabir Samsi (kas499)

Introduction

Graph Convolutional Networks (GCNs) are commonly used for node classification because they learn from both node features and graph structure. However, in homophilic graphs, where connected nodes often share similar labels, the full complexity of a GCN may not always be necessary. Simpler models can preserve the useful neighborhood aggregation step of GCNs by removing additional components, such as nonlinear activations across multiple layers, while still achieving similar performance. In this project, we analyzed the paper “Simplifying Graph Convolutional Networks” by Felix Wu, Amauri Souza, Tianyi Zhang, Christopher Fifty, Tao Yu, and Kilian Weinberger. The paper proposes Simple Graph Convolution, or SGC, which simplifies the standard GCN by keeping the repeated graph propagation step that smooths node features across the graph, while removing nonlinear activation functions and collapsing the model into a single linear classifier. The paper argues that much of GCNs’ predictive power comes from graph propagation rather than nonlinear activations between layers. By removing nonlinearities, SGC achieves performance comparable to that of traditional GCNs on citation and social network datasets while considerably reducing training time.

Chosen Result

We aimed to reproduce the results in Table 2 (see Appendix: Figure 1), which compares GCN and SGC on the Cora, Citeseer, and Pubmed (Sen et al. citation network datasets). Table 2 is the paper’s central empirical result because it supports the main claim that a linear model with precomputed graph propagation can achieve accuracy comparable to a nonlinear GCN while being much faster. We focused on the SGC-GCN comparison, as it represents the paper’s core idea and tests whether GCN nonlinearities are necessary for classification on homophilic graphs.

To further evaluate the efficiency claim, we also referenced Figure 3 (see Appendix: Figure 2), which compares test accuracy and training time on Pubmed and Reddit. Inspired by this analysis, we extended the training time comparison to all three datasets and focused specifically on the runtime difference between SGC and GCN. Together, these experiments allowed us to evaluate both the accuracy and efficiency claims made in the paper.

Methodology

We evaluated both models on the paper’s citation network datasets: Cora, Citeseer, and Pubmed. These were loaded via PyTorch Geometric’s `Planetoid` class with row-normalized node features, using the standard public splits from Sen et al. (2008) of 20 training nodes per class, 500 validation nodes, and 1000 test nodes.

For SGC, we implemented the model from scratch in PyTorch. The preprocessing step computes $X = S^2 X$ once

before training, where $S = \tilde{D}^{-\frac{1}{2}} \tilde{A} \tilde{D}^{-\frac{1}{2}}$ is the normalized adjacency matrix with added self-loops. We constructed the sparse adjacency matrix manually using PyTorch Geometric’s `add_self_loops` and degree utilities, then applied `torch.sparsemm` twice to propagate features across the graph. The model itself is a single linear layer trained with Adam at a learning rate of 0.2 for 100 epochs, or 150 for Citeseer which required more epochs to converge due to its sparser graph structure. We tuned weight decay per dataset via a grid search over 7 log-spaced values from $1e-7$ to $5e-4$, evaluated on the validation set with 3 seeds, and selected the value with the highest mean validation accuracy.

For GCN, we used PyTorch Geometric’s `GCNConv` layers, following the architecture of Kipf & Welling (2017). It contains two graph convolutional layers with a hidden dimension of 16, ReLU activation between layers, and a dropout

of 0.5. GCN was trained with Adam at a learning rate of 0.01 and a fixed weight decay of $5e-4$ for up to 200 epochs, with early stopping based on validation loss with a patience of 10 epochs to prevent overfitting.

Both models were evaluated using test accuracy reported as mean $\pm$ standard deviation over 10 random seeds, with the best model selected based on the highest validation accuracy across all epochs rather than the final epoch. Training time was measured using `time.perf_counter()` and includes the precompute step for SGC to ensure a fair comparison. The main deviation from the paper’s methodology is in weight decay tuning. The original paper uses hyperopt for 60 iterations, while we used a coarser grid search over 7 values with 3 seeds. We also fixed GCN’s weight decay at $5e-4$ following the original Kipf & Welling paper.

Results & Analysis

The results, shown in Appendix: *Table 1*, generally support the findings of the paper by showing that our SGC implementation achieves performance comparable to GCN while using a simpler architecture. We averaged performance across ten runs on the citation datasets, and SGC stayed within around 1% of GCN on Cora and PubMed while slightly outperforming GCN on CiteSeer, reaching 71.9% test accuracy compared to 70.8% for GCN. These support the paper’s argument that much of GCN’s predictive power comes from graph propagation rather than nonlinear transformations between layers. The timing results also support the paper’s efficiency claims. SGC trained roughly 1.5 to 2.8 times faster than GCN on CPU across the citation datasets. Although this speedup is smaller than the paper’s reported $28\times$ GPU speedup, the difference is expected as the original experiments used a GTX 1080 Ti GPU compared to our CPU. The learning curves further highlight the optimization differences between the models. Across Cora, CiteSeer, and PubMed, SGC’s training loss decreased faster and its accuracy stabilized earlier because the model precomputes graph propagation and only trains a linear classifier. In contrast, GCN trained more gradually because it learned multiple nonlinear layers, though it sometimes continued improving over additional epochs. These results suggest that while SGC works well when neighborhood smoothing is sufficient, the additional nonlinear transformations in GCN can still help capture more complex graph patterns.

Independent Exploration

We extended the benchmark to the Actor co-occurrence network, a heterophilic graph where connected nodes often belong to different classes. Unlike the homophilic citation datasets used in the paper, this tests whether SGC’s neighborhood smoothing remains effective when neighboring nodes are dissimilar. Both models performed poorly on Actor, with SGC reaching about 27% test accuracy and GCN about 29%, only slightly above the 20% random baseline. Prior work shows that simple MLP models outperform both methods here, suggesting that neighborhood aggregation can actually hurt performance on heterophilic graphs by weakening class-discriminative information. Even so, the relative pattern between the models was consistent with the citation benchmarks: SGC stayed within $\sim 2\%$ of GCN while training in less than half the time. These results help clarify the limits of the paper’s claim. SGC remains nearly lossless relative to GCN, but both models struggle when neighborhood smoothing is not a useful inductive bias.

Reflections

Re-implementing SGC from scratch revealed that small decisions not specified in the paper, such as including precompute time in runtime comparisons, handling zero-degree nodes, and selecting models by peak validation accuracy, meaningfully affect results. Hyperparameter tuning also proved more consequential than expected, particularly weight decay on small datasets like Citeseer. Overall, our results confirmed the paper’s core claim that SGC matches GCN accuracy while being computationally simpler and faster, and the project deepened our understanding of graph neural networks and reproducibility challenges in machine learning research. Future work could evaluate SGC on much larger graphs to better test scalability claims, run GPU benchmarking to verify the reported $28\times$ speedup, and explore replacing logistic regression with a small MLP on top of S^2X to recover accuracy on harder tasks while preserving most of the efficiency advantage.

Appendix

Table 2. Test accuracy (%) averaged over 10 runs on citation networks. [†]We remove the outliers (accuracy < 75/65/75%) when calculating their statistics due to high variance.

	Cora	Citeseer	Pubmed
Numbers from literature:			
GCN	81.5	70.3	79.0
GAT	83.0 $\pm$ 0.7	72.5 $\pm$ 0.7	79.0 $\pm$ 0.3
GLN	81.2 $\pm$ 0.1	70.9 $\pm$ 0.1	78.9 $\pm$ 0.1
AGNN	83.1 $\pm$ 0.1	71.7 $\pm$ 0.1	79.9 $\pm$ 0.1
LNet	79.5 $\pm$ 1.8	66.2 $\pm$ 1.9	78.3 $\pm$ 0.3
AdaLNet	80.4 $\pm$ 1.1	68.7 $\pm$ 1.0	78.1 $\pm$ 0.4
DeepWalk	70.7 $\pm$ 0.6	51.4 $\pm$ 0.5	76.8 $\pm$ 0.6
DGI	82.3 $\pm$ 0.6	71.8 $\pm$ 0.7	76.8 $\pm$ 0.6
Our experiments:			
GCN	81.4 $\pm$ 0.4	70.9 $\pm$ 0.5	79.0 $\pm$ 0.4
GAT	83.3 $\pm$ 0.7	72.6 $\pm$ 0.6	78.5 $\pm$ 0.3
FastGCN	79.8 $\pm$ 0.3	68.8 $\pm$ 0.6	77.4 $\pm$ 0.3
GIN	77.6 $\pm$ 1.1	66.1 $\pm$ 0.9	77.0 $\pm$ 1.2
LNet	80.2 $\pm$ 3.0 [†]	67.3 $\pm$ 0.5	78.3 $\pm$ 0.6 [†]
AdaLNet	81.9 $\pm$ 1.9 [†]	70.6 $\pm$ 0.8 [†]	77.8 $\pm$ 0.7 [†]
DGI	82.5 $\pm$ 0.7	71.6 $\pm$ 0.7	78.4 $\pm$ 0.7
SGC	81.0 $\pm$ 0.0	71.9 $\pm$ 0.1	78.9 $\pm$ 0.0

Figure 1: ‘Table 2’ of the Paper’s Results

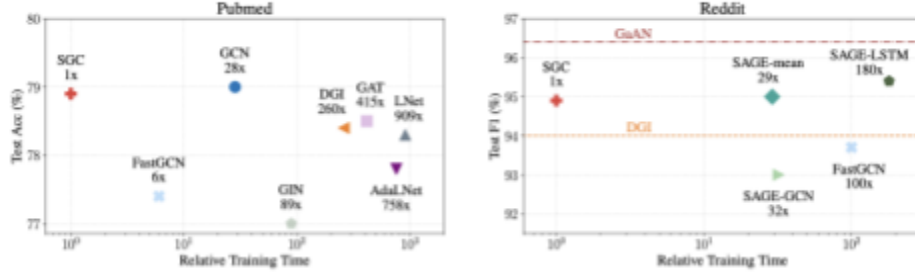


Figure 3. Performance over training time on Pubmed and Reddit. SGC is the fastest while achieving competitive performance. We are not able to benchmark the training time of GaAN and DGI on Reddit because the implementations are not released.

Figure 2: ‘Figure 3’ of the Paper’s Results

Model	Dataset	Test Accuracy (%)	Train Time (s)
SGC	Cora	79.9 $\pm$ 0.2	0.274 $\pm$ 0.004
SGC	Citeseer	71.9 $\pm$ 0.2	0.516 $\pm$ 0.034
SGC	Pubmed	78.7 $\pm$ 0.1	0.321 $\pm$ 0.019
GCN	Cora	81.6 $\pm$ 0.6	0.763 $\pm$ 0.091
GCN	Citeseer	70.8 $\pm$ 0.8	0.794 $\pm$ 0.032
GCN	Pubmed	78.9 $\pm$ 0.7	0.693 $\pm$ 0.077

Table 1: Test Accuracy and Training Time of SGC and GCN for all 3 citation networks

References

Original Paper:

Wu, F., Zhang, T., de Souza Jr., A. H., Fifty, C., Yu, T., & Weinberger, K. Q. (2019). *Simplifying Graph Convolutional Networks*. arXiv preprint arXiv:1902.07153. <https://doi.org/10.48550/arXiv.1902.07153>

Additional References:

Kipf, T. N., & Welling, M. (2017). *Semi-supervised classification with graph convolutional networks*. International Conference on Learning Representations (ICLR 2017). <https://doi.org/10.48550/arXiv.1609.02907>

Pei, H., Wei, B., Chang, K. C. C., Lei, Y., & Yang, B. (2020). Geom-GCN: Geometric Graph Convolutional Networks. *International Conference on Learning Representations (ICLR 2020)*. <https://arxiv.org/abs/2002.05287>

Sen, P., Namata, G., Bilgic, M., Getoor, L., Galligher, B., and Eliassi-Rad, T. *Collective classification in network data*. AI magazine, 29(3):93, 2008.

Frameworks and Tools:

Fey, M., & Lenssen, J. E. (2019). *Fast graph representation learning with PyTorch Geometric*. arXiv preprint arXiv:1903.02428.

Google. (2023). Google Colaboratory. Retrieved from <https://colab.research.google.com>

Harris, C. R., Millman, K. J., van der Walt, S. J., et al. (2020). *Array programming with NumPy*. Nature, 585(7825), 357–362.

Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90–95.

Paszke, A., Gross, S., Massa, F., et al. (2019). *PyTorch: An imperative style, high-performance deep learning library*. Advances in Neural Information Processing Systems, 32.

Github Repo Link: <https://github.com/KabirSamsi/sgc-implementation>